VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum, Karnataka -590014.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Pracheth Vikas Nayak (1BF24CS220)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Pracheth Vikas Nayak (1BF24CS220), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Anusha V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Lab 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS

```
#include <stdio.h>
int main() {
   int n, bt[20], at[20], wt[20], tat[20], ct[20], rt[20], st[20], i;
   float avwt = 0, avtat = 0, avrt = 0;
   printf("Enter total number of processes: ");
   scanf("%d", &n);

   printf("Enter Process Arrival Time:\n");
   for(i = 0; i < n; i++) {
      printf("P[%d]: ", i+1);
      scanf("%d", &at[i]);
   }

   printf("Enter Process Burst Time:\n");
   for(i = 0; i < n; i++) {
      printf("P[%d]: ", i+1);
      scanf("%d", &bt[i]);
   }



   st[0] = at[0];
   ct[0] = st[0] + bt[0];
   tat[0] = ct[0] - at[0];
   wt[0] = tat[0] - bt[0];
   rt[0] = st[0] - at[0];
```

```
    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i]) {
            st[i] = at[i];
        } else {
            st[i] = ct[i-1];
        }
        ct[i] = st[i] + bt[i];
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = st[i] - at[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT");
    for(i = 0; i < n; i++) {
        avwt += wt[i];
        avtat += tat[i];
        avrt += rt[i];
        printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\n\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);

    return 0;
}
```

OUTPUT:

```
Enter total number of processes: 4
Enter Process Arrival Time:
P[1]: 0
P[2]: 3
P[3]: 5
P[4]: 7
Enter Process Burst Time:
P[1]: 2
P[2]: 5
P[3]: 6
P[4]: 8

Process AT      BT      CT      TAT     WT      RT
P[1]    0       2       2       2       0       0
P[2]    3       5       8       5       0       0
P[3]    5       6       14      9       3       3
P[4]    7       8       22      15      7       7

Average Waiting Time: 2.50
Average Turnaround Time: 7.75
Average Response Time: 2.50

Process returned 0 (0x0)   execution time : 53.445 s
Press any key to continue.
```

b) SJF non pre-emptive

```
#include <stdio.h>

int main() {
    int n, bt[20], at[20], wt[20], tat[20], ct[20], rt[20], st[20], completed[20];
    int i, time = 0, completedCount = 0;
    float avwt = 0, avtat = 0, avrt = 0;

    printf("SJF (Non-Preemptive) CPU Scheduling\n");
    printf("Enter total number of processes: ");
    scanf("%d", &n);
```

```
printf("Enter Process Arrival Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &at[i]);
}

printf("Enter Process Burst Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &bt[i]);
   completed[i] = 0;
}

while(completedCount < n) {
   int idx = -1, minBT = 9999;
   for(i = 0; i < n; i++) {
      if(at[i] <= time && completed[i] == 0 && bt[i] < minBT) {
         minBT = bt[i];
         idx = i;
      }
   }
   if(idx == -1) {
      time++;
   } else {
      st[idx] = time;
      time += bt[idx];
      ct[idx] = time;
      tat[idx] = ct[idx] - at[idx];
      wt[idx] = tat[idx] - bt[idx];
      rt[idx] = st[idx] - at[idx];
      completed[idx] = 1;
```

```
            completedCount++;
        }
    }

    printf("\nProcess\tAT\tBT\tST\tCT\tTAT\tWT\tRT");
    for(i = 0; i < n; i++) {
        avwt += wt[i];
        avtat += tat[i];
        avrt += rt[i];
        printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], st[i], ct[i], tat[i],
wt[i], rt[i]);
    }

    printf("\n\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);

    return 0;
}
```

OUTPUT:

```
SJF (Non-Preemptive) CPU Scheduling
Enter total number of processes: 4
Enter Process Arrival Time:
P[1]: 0
P[2]: 3
P[3]: 5
P[4]: 7
Enter Process Burst Time:
P[1]: 2
P[2]: 5
P[3]: 6
P[4]: 8

Process AT      BT      ST      CT      TAT     WT      RT
P[1]    0       2       0       2       2       0       0
P[2]    3       5       3       8       5       0       0
P[3]    5       6       8       14      9       3       3
P[4]    7       8       14      22      15      7       7

Average Waiting Time: 2.50
Average Turnaround Time: 7.75
Average Response Time: 2.50

Process returned 0 (0x0)   execution time : 31.267 s
Press any key to continue.
```

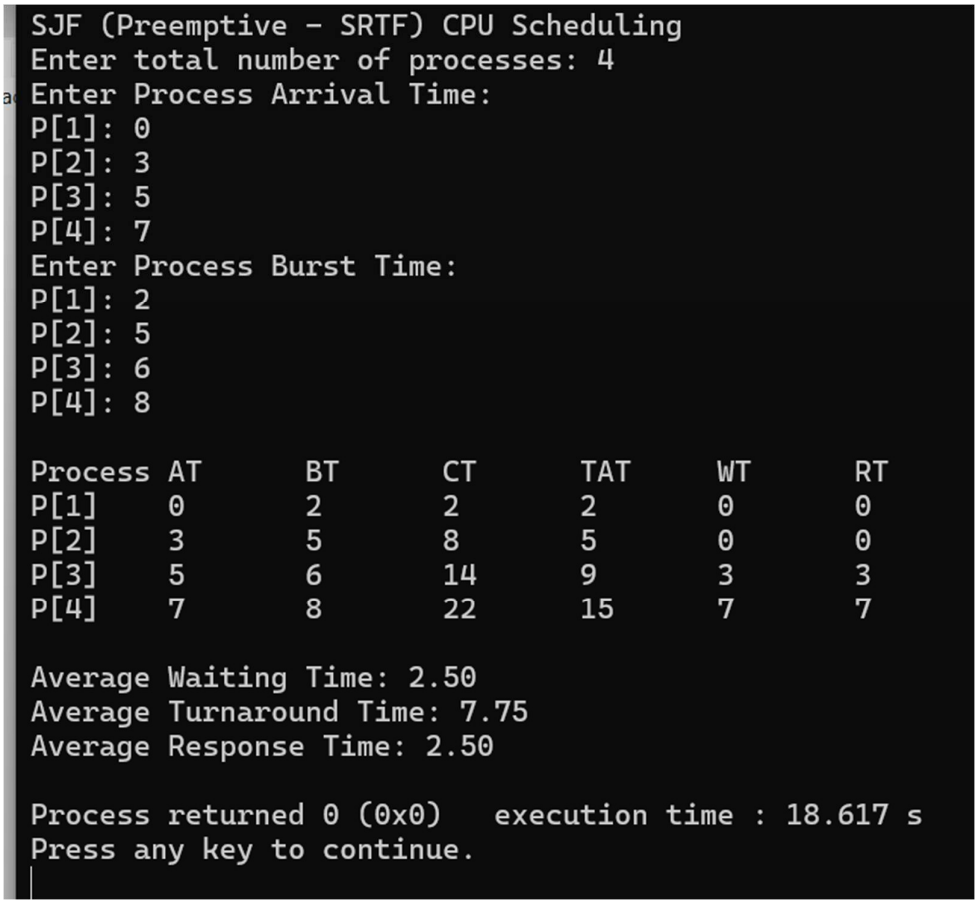
SJF pre-emptive

```
#include <stdio.h>

int main() {
   int n, bt[20], at[20], rtRemaining[20], wt[20], tat[20], ct[20], rt[20], st[20],
firstResponse[20];
   int i, time = 0, completedCount = 0;
   float avwt = 0, avtat = 0, avrt = 0;

   printf("SJF (Preemptive - SRTF) CPU Scheduling\n");
   printf("Enter total number of processes: ");
   scanf("%d", &n);

   printf("Enter Process Arrival Time:\n");
   for(i = 0; i < n; i++) {
      printf("P[%d]: ", i+1);
      scanf("%d", &at[i]);
   }

   printf("Enter Process Burst Time:\n");
   for(i = 0; i < n; i++) {
      printf("P[%d]: ", i+1);
      scanf("%d", &bt[i]);
      rtRemaining[i] = bt[i];
      firstResponse[i] = -1;
   }

   while(completedCount < n) {
      int idx = -1, minRT = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && rtRemaining[i] > 0 && rtRemaining[i] < minRT) {
```

```
            minRT = rtRemaining[i];
            idx = i;
        }
    }
    if(idx == -1) {
        time++;
    } else {
        if(firstResponse[idx] == -1) {
            st[idx] = time;
            firstResponse[idx] = time;
        }
        rtRemaining[idx]--;
        time++;
        if(rtRemaining[idx] == 0) {
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            rt[idx] = st[idx] - at[idx];
            completedCount++;
        }
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT");
for(i = 0; i < n; i++) {
    avwt += wt[i];
    avtat += tat[i];
    avrt += rt[i];
    printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], ct[i], tat[i], wt[i],
rt[i]);
}
```

```
    printf("\n\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);

    return 0;
}
```

OUTPUT:



c) Priority non-pre-emptive

```
#include <stdio.h>
int main() {
    int n, at[20], bt[20], pr[20], ct[20], tat[20], wt[20], rt[20], st[20], completed[20];
    int i, time = 0, completedCount = 0;
```

```
float avwt = 0, avtat = 0, avrt = 0;
printf("Priority (Non-Preemptive) CPU Scheduling\n");
printf("Enter total number of processes: ");
scanf("%d", &n);
  printf("Enter Process Arrival Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &at[i]);
}

printf("Enter Process Burst Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &bt[i]);
}

   printf("Enter Process Priority (Lower number = Higher priority): ");
   for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &pr[i]);
   completed[i] = 0;
}
while(completedCount < n) {
   int idx = -1, minPr = 9999;

   for(i = 0; i < n; i++) {
      if(at[i] <= time && completed[i] == 0 && pr[i] < minPr) {
         minPr = pr[i];
         idx = i;
      }
   }
```

```
        if(idx == -1) {
            time++;
        } else {
            st[idx] = time;
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            rt[idx] = st[idx] - at[idx];
            completed[idx] = 1;
            completedCount++;
        }
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT");
    for(i = 0; i < n; i++) {
        avwt += wt[i];
        avtat += tat[i];
        avrt += rt[i];
        printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], pr[i], ct[i], tat[i],
wt[i], rt[i]);
    }
    printf("\n\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);
    return 0;
}
```

OUTPUT:

```
Priority (Non-Preemptive) CPU Scheduling
Enter total number of processes: 3
Enter Process Arrival Time:
P[1]: 3
P[2]: 4
P[3]: 2
Enter Process Burst Time:
P[1]: 5
P[2]: 7
P[3]: 6
Enter Process Priority (Lower number = Higher priority): P[1]: 2
P[2]: 1
P[3]: 3

Process AT      BT      PR      CT      TAT     WT      RT
P[1]    3       5       2       20      17      12      12
P[2]    4       7       1       15      11      4       4
P[3]    2       6       3       8       6       0       0

Average Waiting Time: 5.33
Average Turnaround Time: 11.33
Average Response Time: 5.33
```

Priority pre-emptive

```
#include <stdio.h>

int main() {
    int n, at[20], bt[20], pr[20], rtRemaining[20], ct[20], tat[20], wt[20], rt[20], st[20], firstResponse[20];
    int i, time = 0, completedCount = 0;
    float avwt = 0, avtat = 0, avrt = 0;
    printf("Priority (Preemptive) CPU Scheduling\n");
    printf("Enter total number of processes: ");
    scanf("%d", &n);
    printf("Enter Process Arrival Time:\n");
    for(i = 0; i < n; i++) {
        printf("P[%d]: ", i+1);
        scanf("%d", &at[i]);
    }
    printf("Enter Process Burst Time:\n");
```

```
for(i = 0; i < n; i++) {
    printf("P[%d]: ", i+1);
    scanf("%d", &bt[i]);
}
    printf("Enter Process Priority (Lower number = Higher priority): ");
    for(i = 0; i < n; i++) {
    printf("P[%d]: ", i+1);
    scanf("%d", &pr[i]);
    rtRemaining[i] = bt[i];
    firstResponse[i] = -1;
}
while(completedCount < n) {
    int idx = -1, minPr = 9999;
    for(i = 0; i < n; i++) {
        if(at[i] <= time && rtRemaining[i] > 0 && pr[i] < minPr) {
            minPr = pr[i];
            idx = i;
        }
    }
    if(idx == -1) {
        time++;
    } else {
        if(firstResponse[idx] == -1) {
            st[idx] = time;
            firstResponse[idx] = time;
        }
        rtRemaining[idx]--;
        time++;

        if(rtRemaining[idx] == 0) {
            ct[idx] = time;
```

```
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            rt[idx] = st[idx] - at[idx];
            completedCount++;
          }
        }
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT");
    for(i = 0; i < n; i++) {
        avwt += wt[i];
        avtat += tat[i];
        avrt += rt[i];
        printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], pr[i], ct[i], tat[i],
wt[i], rt[i]);
    }
    printf("\n\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);

    return 0;
}
```

OUTPUT:

```
Priority (Preemptive) CPU Scheduling
Enter total number of processes: 3
Enter Process Arrival Time:
P[1]: 3
P[2]: 4
P[3]: 2
Enter Process Burst Time:
P[1]: 5
P[2]: 7
P[3]: 6
Enter Process Priority (Lower number = Higher priority): P[1]: 2
P[2]: 1
P[3]: 3

Process AT      BT      PR      CT      TAT     WT      RT
P[1]    3       5       2       15      12      7       0
P[2]    4       7       1       11      7       0       0
P[3]    2       6       3       20      18      12      0

Average Waiting Time: 6.33
Average Turnaround Time: 12.33
Average Response Time: 0.00
```

d) Round robin

```
#include <stdio.h>



int main() {
    int n, tq;
    int at[20], bt[20], rtRemaining[20], ct[20], tat[20], wt[20], rt[20], firstResponse[20];
    int queue[1000], front = 0, rear = 0;
    int inQueue[20] = {0};
    int i, time = 0, completedCount = 0;
    float avwt = 0, avtat = 0, avrt = 0;

    printf("Round Robin (RR) CPU Scheduling\n");
    printf("Enter total number of processes: ");
    scanf("%d", &n);
```

```
printf("Enter the Time Quantum: ");
scanf("%d", &tq);

printf("Enter Process Arrival Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &at[i]);
}

printf("Enter Process Burst Time:\n");
for(i = 0; i < n; i++) {
   printf("P[%d]: ", i+1);
   scanf("%d", &bt[i]);
   rtRemaining[i] = bt[i];
   firstResponse[i] = -1;
}
int min_at = 9999;
for(i = 0; i < n; i++) {
   if(at[i] < min_at)
      min_at = at[i];
}
time = min_at;
for(i = 0; i < n; i++) {
   if(at[i] <= time && !inQueue[i]) {
      queue[rear++] = i;
      inQueue[i] = 1;
   }
}

while(completedCount < n) {
```

```
if(front == rear) {
    time++;
    for(i = 0; i < n; i++) {
        if(at[i] <= time && !inQueue[i] && rtRemaining[i] > 0) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }
    }
    continue;
}

int idx = queue[front++];
if(firstResponse[idx] == -1) {
    firstResponse[idx] = time;
    rt[idx] = firstResponse[idx] - at[idx];
}

int execTime = (rtRemaining[idx] > tq) ? tq : rtRemaining[idx];
rtRemaining[idx] -= execTime;
time += execTime;
for(i = 0; i < n; i++) {
    if(at[i] <= time && !inQueue[i] && rtRemaining[i] > 0) {
        queue[rear++] = i;
        inQueue[i] = 1;
    }
}

if(rtRemaining[idx] > 0) {
    queue[rear++] = idx;
} else {
    ct[idx] = time;
```

```
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            completedCount++;
        }
    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT");
    for(i = 0; i < n; i++) {
        avwt += wt[i];
        avtat += tat[i];
        avrt += rt[i];
        printf("\nP[%d]\t%d\t%d\t%d\t%d\t%d\t%d", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\n\nTime Quantum used: %d", tq);
    printf("\nAverage Waiting Time: %.2f", avwt/n);
    printf("\nAverage Turnaround Time: %.2f", avtat/n);
    printf("\nAverage Response Time: %.2f\n", avrt/n);

    return 0;
}
```

OUTPUT:

```
Round Robin (RR) CPU Scheduling
Enter total number of processes: 3
Enter the Time Quantum: 3
Enter Process Arrival Time:
P[1]: 3
P[2]: 4
P[3]: 2
Enter Process Burst Time:
P[1]: 5
P[2]: 7
P[3]: 6

Process AT      BT      CT      TAT     WT      RT
P[1]    3       5       16      13      8       2
P[2]    4       7       20      16      9       4
P[3]    2       6       14      12      6       0

Time Quantum used: 3
Average Waiting Time: 7.67
Average Turnaround Time: 13.67
Average Response Time: 2.00
```

```
Round Robin (RR) CPU Scheduling
Enter total number of processes: 3
Enter the Time Quantum: 5
Enter Process Arrival Time:
P[1]: 3
P[2]: 4
P[3]: 2
Enter Process Burst Time:
P[1]: 5
P[2]: 7
P[3]: 6

Process AT      BT      CT      TAT     WT      RT
P[1]    3       5       12      9       4       4
P[2]    4       7       20      16      9       8
P[3]    2       6       18      16      10      0

Time Quantum used: 5
Average Waiting Time: 7.67
Average Turnaround Time: 13.67
Average Response Time: 4.00
```

LAB 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

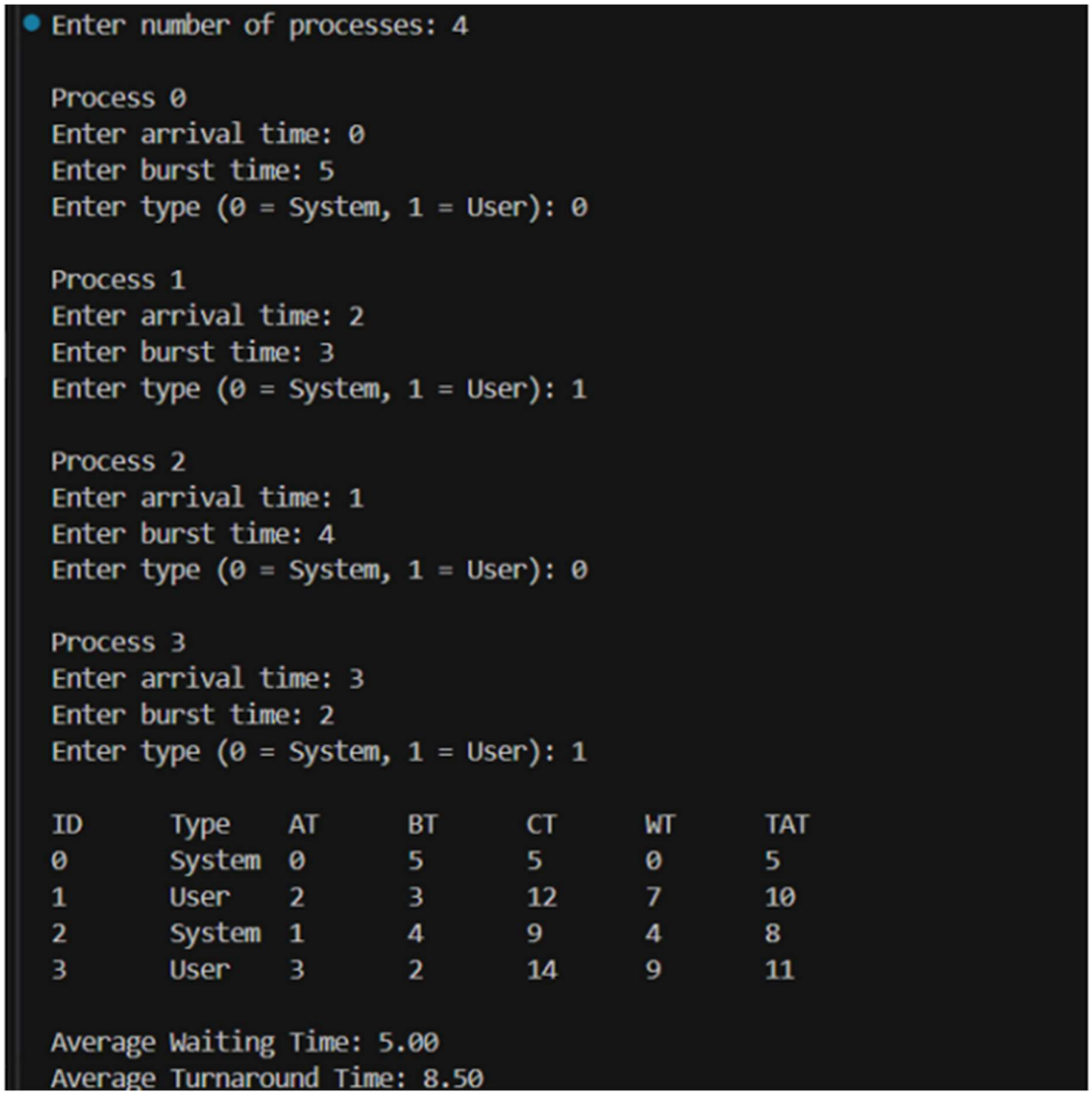
```
#include <stdio.h>
#include <string.h>

struct Process {
    int id;
    int at;
    int bt;
    int rt;
    int type;
    int ct;
    int wt;
    int tat;
};
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i;
        printf("\nProcess %d\n", i);
        printf("Enter arrival time: ");
        scanf("%d", &p[i].at);
        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);
        printf("Enter type (0 = System, 1 = User): ");
```

```
    scanf("%d", &p[i].type);

    p[i].rt = p[i].bt;
}
int current_time = 0;
int completed = 0;
while (completed < n) {
    int selected_process = -1;
    int best_sys = -1;
    int best_sys_at = 999999;
    int best_usr = -1;
    int best_usr_at = 999999;

    for (int i = 0; i < n; i++) {
        if (p[i].rt > 0 && p[i].at <= current_time) {
            if (p[i].type == 0) {
                if (p[i].at < best_sys_at) {
                    best_sys_at = p[i].at;
                    best_sys = i;
                }
            } else {
                if (p[i].at < best_usr_at) {
                    best_usr_at = p[i].at;
                    best_usr = i;
                }
            }
        }
    }
    if (best_sys != -1) {
        selected_process = best_sys;
    } else if (best_usr != -1) {
```

```
            selected_process = best_usr;
        }

        if (selected_process != -1) {
            p[selected_process].rt--;
            current_time++;

            if (p[selected_process].rt == 0) {
                p[selected_process].ct = current_time;
                p[selected_process].tat = p[selected_process].ct - p[selected_process].at;
                p[selected_process].wt = p[selected_process].tat - p[selected_process].bt;
                completed++;
            }
        } else {
            current_time++;
        }
    }
    printf("\n%-4s %-9s %-5s %-5s %-5s %-5s %-5s\n", "ID", "Type", "AT", "BT", "CT", "WT",
"TAT");
    float total_wt = 0, total_tat = 0;
    for (int i = 0; i < n; i++) {
        char type_str[10];
        if (p[i].type == 0) {
            strcpy(type_str, "System");
        } else {
            strcpy(type_str, "User");
        }
        printf("%-4d %-9s %-5d %-5d %-5d %-5d %-5d\n",
               p[i].id, type_str, p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }
```

```
    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);

    return 0;
}
```

OUTPUT:



LAB 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>

#define MAX 10

typedef struct {
   int id, bt, dl, per, ct, wt, tat;
} Process;

void edf(Process p[], int n) {
   printf("\n====== Earliest Deadline First (EDF) Scheduling ======\n");

   double util = 0.0;
   for (int i = 0; i < n; i++)
      util += (double)p[i].bt / p[i].dl;

   printf("CPU Utilization: %.2f\n", util);
   printf(util <= 1.0 ? "Schedulable (Utilization <= 1)\n" : "NOT Schedulable (Utilization > 1)\n");

   Process s[MAX];
   for (int i = 0; i < n; i++) s[i] = p[i];

   for (int i = 0; i < n - 1; i++)
      for (int j = i + 1; j < n; j++)
         if (s[i].dl > s[j].dl) {
            Process tmp = s[i]; s[i] = s[j]; s[j] = tmp;
         }
```

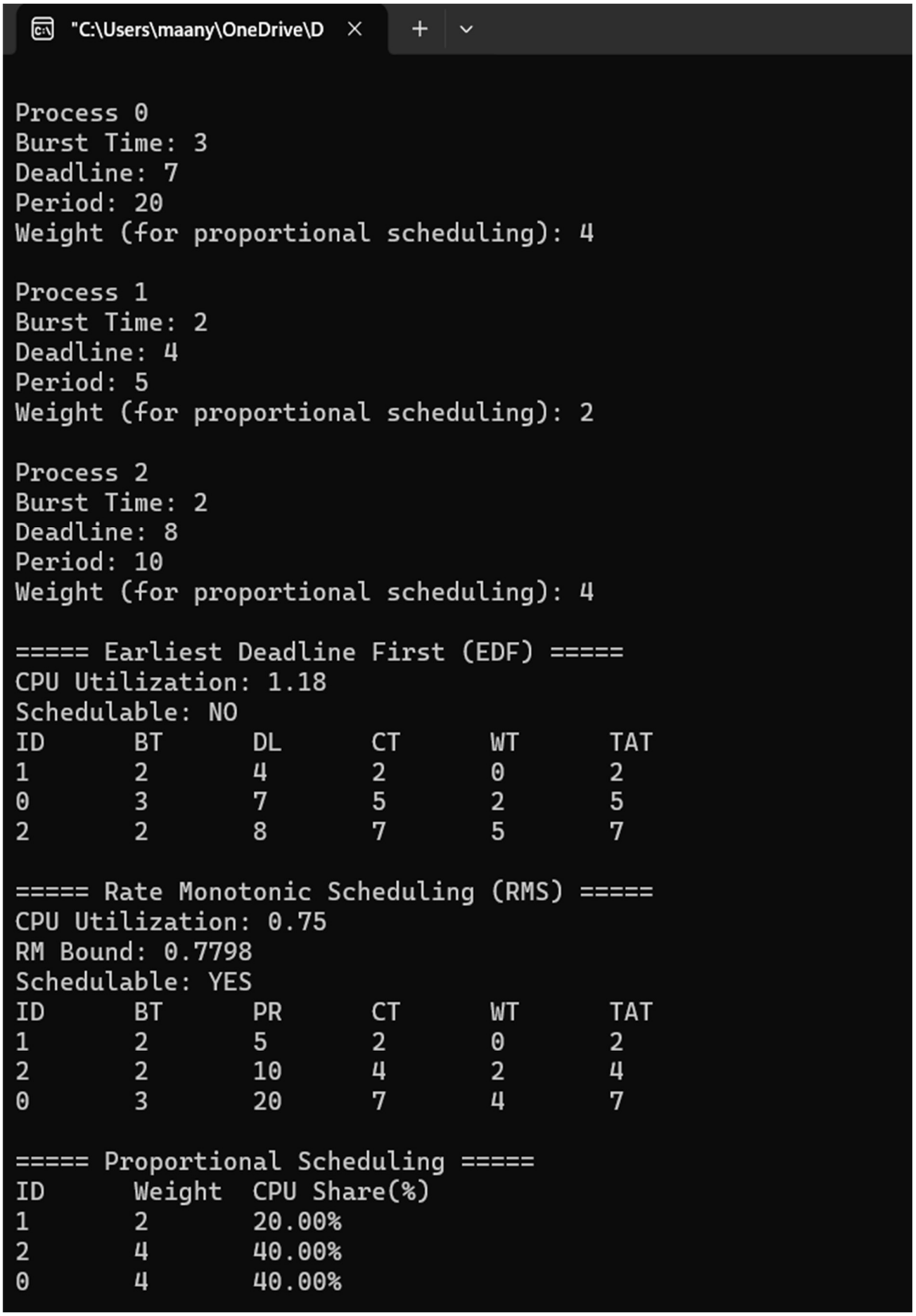
```
    printf("%-4s %-4s %-10s %-4s %-4s %-4s\n", "ID", "BF", "Deadline", "CT", "WT",
"TAT");

    int t = 0;
    for (int i = 0; i < n; i++) {
        t += s[i].bt;
        s[i].ct  = t;
        s[i].tat = s[i].ct;
        s[i].wt  = s[i].tat - s[i].bt;
        printf("%-4d %-4d %-10d %-4d %-4d %-4d\n",
               s[i].id, s[i].bt, s[i].dl, s[i].ct, s[i].wt, s[i].tat);
    }
}

void rms(Process p[], int n) {
    printf("\n====== Rate Monotonic Scheduling (RMS) ======\n");

    double util = 0.0;
    for (int i = 0; i < n; i++)
        util += (double)p[i].bt / p[i].per;

    double rmBound = n * (pow(2.0, 1.0 / n) - 1.0);

    printf("CPU Utilization: %.2f\n", util);
    printf("RM Bound: %.4f\n", rmBound);
    printf(util <= rmBound ? "Schedulable (Utilization <= RM Bound)\n" : "NOT Schedulable
(Utilization > RM Bound)\n");

    Process s[MAX];
    for (int i = 0; i < n; i++) s[i] = p[i];
```

```
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (s[i].per > s[j].per) {
                Process tmp = s[i]; s[i] = s[j]; s[j] = tmp;
            }

    printf("%-4s %-4s %-8s %-4s %-4s %-4s\n", "ID", "BF", "Period", "CT", "WT", "TAT");

    int t = 0;
    for (int i = 0; i < n; i++) {
        t += s[i].bt;
        s[i].ct  = t;
        s[i].tat = s[i].ct;
        s[i].wt  = s[i].tat - s[i].bt;
        printf("%-4d %-4d %-8d %-4d %-4d %-4d\n",
               s[i].id, s[i].bt, s[i].per, s[i].ct, s[i].wt, s[i].tat);
    }
}

void proportional(Process p[], int n) {
    printf("\n====== Proportional Scheduling ======\n");

    int totBt = 0, hyper = 1;
    for (int i = 0; i < n; i++) { totBt += p[i].bt; hyper *= p[i].per; }

    printf("%-4s %-4s %-10s %-12s %-10s\n", "ID", "BF", "Period", "Share(%)",
"Slots/Hyper");

    for (int i = 0; i < n; i++) {
        double share = (double)p[i].bt / totBt * 100.0;
        int slots    = (int)(share / 100.0 * hyper);
        printf("%-4d %-4d %-10d %-12.2f %-10d\n",
```

```
            p[i].id, p[i].bt, p[i].per, share, slots);
    }
    printf("Hyperperiod (product of periods): %d\n", hyper);
}

int main() {
    int n;
    Process p[MAX];

    printf("Real-Time CPU Scheduling\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("\nEnter process details:\n");
    for (int i = 0; i < n; i++) {
        p[i].id = i;
        printf("\nProcess %d:\n", i);
        printf("Burst Time: ");         scanf("%d", &p[i].bt);
        printf("Deadline (for EDF): "); scanf("%d", &p[i].dl);
        printf("Period (for RMS): ");   scanf("%d", &p[i].per);
    }

    edf(p, n);
    rms(p, n);
    proportional(p, n);

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 3

Enter process details:

Process 0:
Burst Time: 2
Deadline (for EDF): 5
Period (for RMS): 5

Process 1:
Burst Time: 2
Deadline (for EDF): 7
Period (for RMS): 7

Process 2:
Burst Time: 3
Deadline (for EDF): 8
Period (for RMS): 8

===== Earliest Deadline First (EDF) Scheduling =====
CPU Utilization: 1.06
ID      BF      Deadline        CT      WT      TAT
0       2       5               2       0       2
1       2       7               4       2       4
2       3       8               7       4       7

===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 1.06
RM Bound: 0.7798
ID      BF      Period  CT      WT      TAT
0       2       5       2       0       2
1       2       7       4       2       4
2       3       8       7       4       7
```


"C:\Users\maany\OneDrive\D
Process 0
Burst Time: 3
Deadline: 7
Period: 20
Weight (for proportional scheduling): 4
Process 1
Burst Time: 2
Deadline: 4
Period: 5
Weight (for proportional scheduling): 2
Process 2
Burst Time: 2
Deadline: 8
Period: 10
Weight (for proportional scheduling): 4
===== Earliest Deadline First (EDF) =====
CPU Utilization: 1.18
Schedulable: NO
ID BT DL CT WT TAT
1 2 4 2 0 2
0 3 7 5 2 5
2 2 8 7 5 7
===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 0.75
RM Bound: 0.7798
Schedulable: YES
ID BT PR CT WT TAT
1 2 5 2 0 2
2 2 10 4 2 4
0 3 20 7 4 7
===== Proportional Scheduling =====
ID Weight CPU Share(%)
1 2 20.00%
2 4 40.00%
0 4 40.00%

LAB 4

Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem

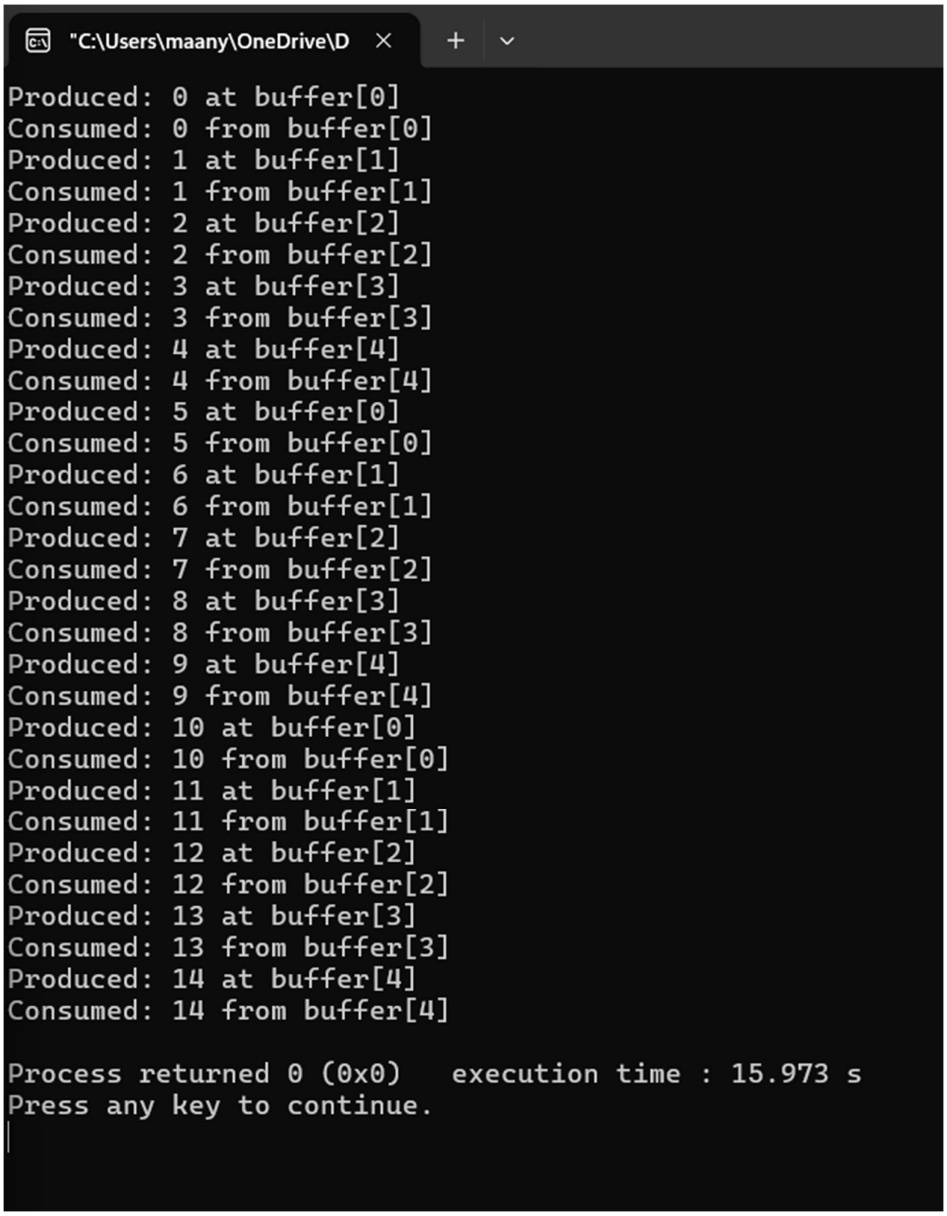
a) Producer-Consumer problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int item_count = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

int total_items = 0;

void* producer(void* arg) {
    for (int i = 0; i < total_items; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item_count;
        printf("Produced: %d at buffer[%d]\n", item_count, in);
        fflush(stdout);
        in = (in + 1) % BUFFER_SIZE;
```

```
        item_count++;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(50000);
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    for (int i = 0; i < total_items; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int consumed_item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", consumed_item, out);
        fflush(stdout);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(75000);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t producer_thread;
    pthread_t consumer_thread;

    printf("Enter number of items to produce/consume: ");
    scanf("%d", &total_items);
    printf("Result:\n");
```

```
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

OUTPUT:



b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
```

```
sem_t forks[N];
sem_t room;

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        sem_wait(&room);
        if (id % 2 == 0) {
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
        } else {
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(1);
        sem_post(&forks[id]);
        sem_post(&forks[(id + 1) % N]);
        printf("Philosopher %d put down forks %d and %d.\n", id, id, (id + 1) % N);
        sem_post(&room);
    }
}

int main() {
```

```
    pthread_t phil[N];
    int ids[N];
    sem_init(&room, 0, N - 1);
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(phil[i], NULL);
    }
    return 0;
}
```

OUTPUT:


"C:\Users\maany\OneDrive\D
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 is thinking.

LAB 5

Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection

a) Bankers' algorithm

```
#include <stdio.h>

int main() {
   int n, m;

   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m], avail[m];

   printf("Enter Allocation Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);

   printf("Enter Maximum Demand Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) {
         scanf("%d", &max[i][j]);
         need[i][j] = max[i][j] - alloc[i][j];
      }

   printf("Enter Available Resources:\n");
   for (int j = 0; j < m; j++)
      scanf("%d", &avail[j]);
```

```
// Safety Algorithm
int finish[n], safeSeq[n];
int work[m];

for (int j = 0; j < m; j++) work[j] = avail[j];
for (int i = 0; i < n; i++) finish[i] = 0;

int count = 0;
while (count < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int canRun = 1;
            for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }
            if (canRun) {
                for (int j = 0; j < m; j++)
                    work[j] += alloc[i][j];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (!found) break;
}
```

```
    printf("\n");
    if (count == n) {
        printf("System is in a safe state.\n");
        printf("Safe sequence is: ");
        for (int i = 0; i < n; i++) {
            printf("P%d", safeSeq[i]);
            if (i < n - 1) printf(" -> ");
        }
        printf("\n");
    } else {
        printf("System is in an unsafe state (Potential Deadlock).\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

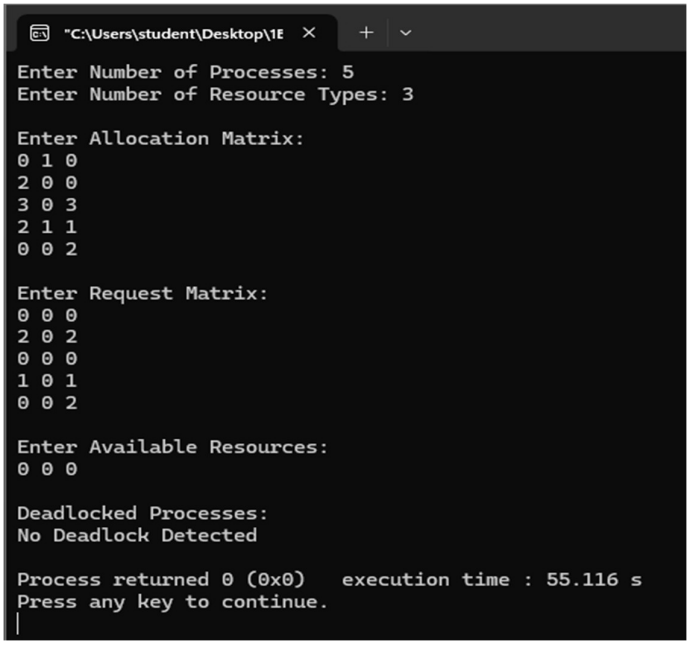
b) Deadlock Detection

```
#include <stdio.h>

int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m], avail[m];

    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Request Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &request[i][j]);

    printf("Enter Available Resources:\n");
    for (int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    // Detection Algorithm
    int finish[n];
    int work[m];

    // Mark processes with zero allocation as finished
```

```
for (int i = 0; i < n; i++) {
    finish[i] = 0;
    int zeroAlloc = 1;
    for (int j = 0; j < m; j++) {
        if (alloc[i][j] != 0) {
            zeroAlloc = 0;
            break;
        }
    }
    if (zeroAlloc) finish[i] = 1;
}

for (int j = 0; j < m; j++) work[j] = avail[j];

int found;
do {
    found = 0;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int canRun = 1;
            for (int j = 0; j < m; j++) {
                if (request[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }
            if (canRun) {
                for (int j = 0; j < m; j++)
                    work[j] += alloc[i][j];
                finish[i] = 1;
                found = 1;
```

```
            }
        }
    }
} while (found);

printf("\nProcesses in Deadlock:\n");
int deadlockFound = 0;
for (int i = 0; i < n; i++) {
    if (!finish[i]) {
        printf("P%d\n", i);
        deadlockFound = 1;
    }
}
if (!deadlockFound)
    printf("No deadlock detected. All processes can finish.\n");

return 0;
}
```

OUTPUT:



LAB 6

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>


void firstFit(int blockSize[], int m, int processSize[], int n) {
   int allocation[n];
   for (int i = 0; i < n; i++)
      allocation[i] = -1;


   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (blockSize[j] >= processSize[i]) {
            allocation[i] = j + 1;
            blockSize[j] -= processSize[i];
            break;
         }
      }
   }


   printf("\n--- First Fit ---\n");
   printf("Process No.\tProcess Size\tBlock No.\n");
   for (int i = 0; i < n; i++) {
      printf("%d\t\t%d\t\t", i + 1, processSize[i]);
      if (allocation[i] != -1)
         printf("%d\n", allocation[i]);
      else
         printf("Not Allocated\n");
   }
}
```

```
void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    int tempBlock[m];

    for (int i = 0; i < m; i++)
        tempBlock[i] = blockSize[i];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlock[j] >= processSize[i]) {
                if (bestIdx == -1 || tempBlock[j] < tempBlock[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx + 1;
            tempBlock[bestIdx] -= processSize[i];
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
```

```
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    int tempBlock[m];

    for (int i = 0; i < m; i++)
        tempBlock[i] = blockSize[i];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlock[j] >= processSize[i]) {
                if (worstIdx == -1 || tempBlock[j] > tempBlock[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx + 1;
            tempBlock[worstIdx] -= processSize[i];
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
```

```
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    // Save original block sizes for each algorithm
    int block1[m], block2[m];
    for (int i = 0; i < m; i++) {
        block1[i] = blockSize[i];
        block2[i] = blockSize[i];
    }
```

```
    firstFit(blockSize, m, processSize, n);
    bestFit(block1, m, processSize, n);
    worstFit(block2, m, processSize, n);

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter sizes of 5 processes:
115 500 358 200 375

--- First Fit ---
Process No.     Process Size    Block No.
1               115             1
2               500             2
3               358             3
4               200             4
5               375             5

--- Best Fit ---
Process No.     Process Size    Block No.
1               115             1
2               500             2
3               358             3
4               200             4
5               375             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               115             2
2               500             Not Allocated
3               358             2
4               200             5
5               375             Not Allocated
```

LAB 7

Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal

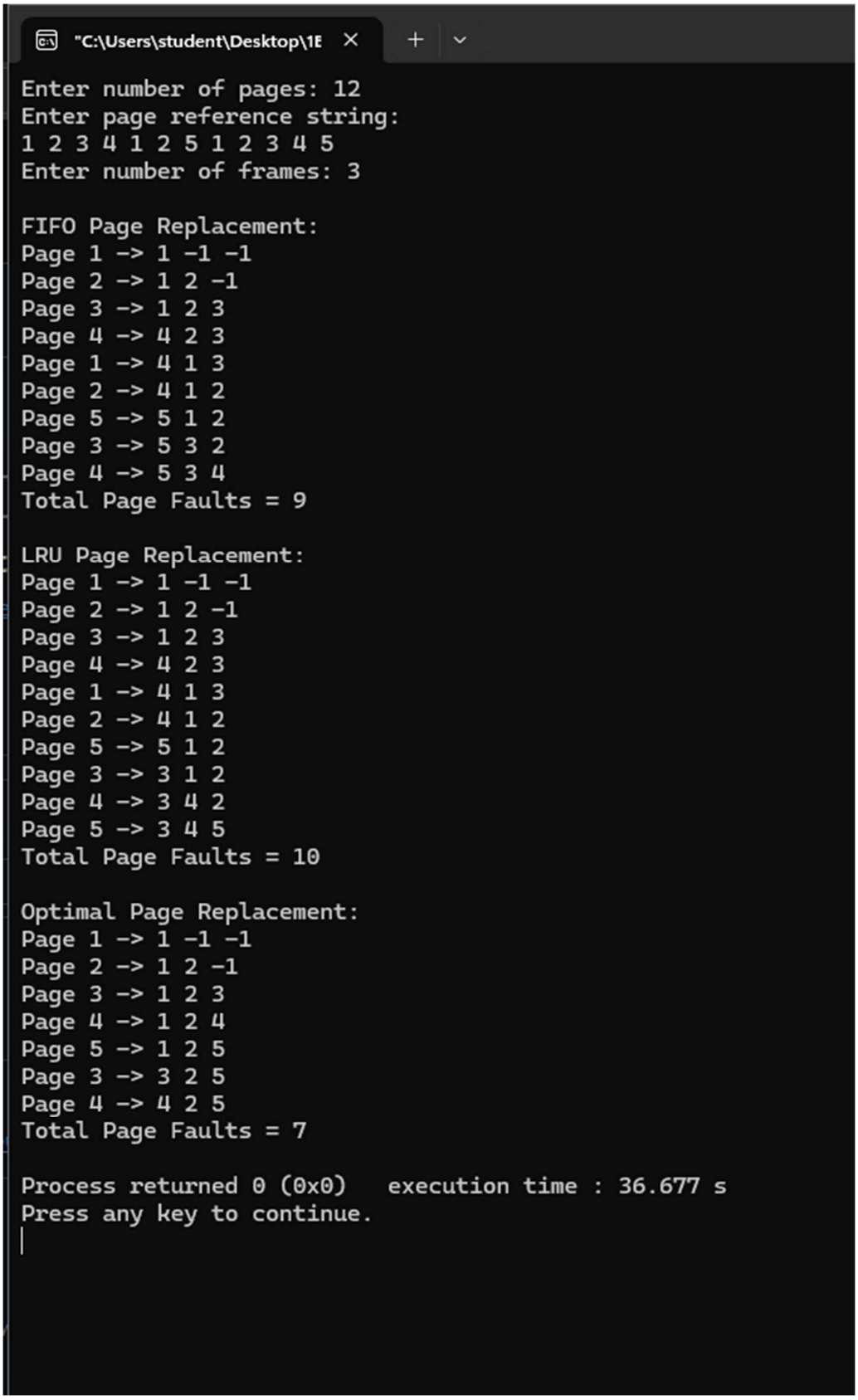
```
#include <stdio.h>
#define MAX 50

int search(int key, int frame[], int frameSize) {
   for (int i = 0; i < frameSize; i++) {
      if (frame[i] == key)
         return i;
   }
   return -1;
}

void FIFO(int pages[], int n, int frameSize) {
   int frame[MAX], faults = 0, index = 0;
   for (int i = 0; i < frameSize; i++)
      frame[i] = -1;
   printf("\nFIFO Page Replacement:\n");
   for (int i = 0; i < n; i++) {
      if (search(pages[i], frame, frameSize) == -1) {
         frame[index] = pages[i];
         index = (index + 1) % frameSize;
         faults++;
         printf("Page %d -> ", pages[i]);
         for (int j = 0; j < frameSize; j++)
            printf("%d ", frame[j]);
         printf("\n");
      }
   }
   printf("Total Page Faults = %d\n", faults);
```

```
}

void LRU(int pages[], int n, int frameSize) {
    int frame[MAX], time[MAX];
    int faults = 0, counter = 0;
    for (int i = 0; i < frameSize; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
    printf("\nLRU Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        int pos = search(pages[i], frame, frameSize);
        if (pos != -1) {
            time[pos] = ++counter;
        } else {
            int lru = 0;
            for (int j = 1; j < frameSize; j++) {
                if (time[j] < time[lru])
                    lru = j;
            }

            frame[lru] = pages[i];
            time[lru] = ++counter;
            faults++;
            printf("Page %d -> ", pages[i]);
            for (int j = 0; j < frameSize; j++)
                printf("%d ", frame[j]);
            printf("\n");
        }
    }
    printf("Total Page Faults = %d\n", faults);
```

```
}

int predict(int pages[], int frame[], int n, int index, int frameSize) {
    int farthest = index, pos = -1;
    for (int i = 0; i < frameSize; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frame[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    pos = i;
                }
                break;
            }
        }
        if (j == n)
            return i;
    }
    return (pos == -1) ? 0 : pos;
}

void Optimal(int pages[], int n, int frameSize) {
    int frame[MAX];
    int faults = 0;
    int filled = 0;
    for (int i = 0; i < frameSize; i++)
        frame[i] = -1;
    printf("\nOptimal Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        if (search(pages[i], frame, frameSize) == -1) {
            if (filled < frameSize) {
```

```
            frame[filled++] = pages[i];
        } else {
            int pos = predict(pages, frame, n, i + 1, frameSize);
            frame[pos] = pages[i];
        }
        faults++;
        printf("Page %d -> ", pages[i]);
        for (int j = 0; j < frameSize; j++)
            printf("%d ", frame[j]);
        printf("\n");
      }
   }
   printf("Total Page Faults = %d\n", faults);
}

int main() {
   int pages[MAX], n, frameSize;
   printf("Enter number of pages: ");
   scanf("%d", &n);
   printf("Enter page reference string:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &pages[i]);
   printf("Enter number of frames: ");
   scanf("%d", &frameSize);
   FIFO(pages, n, frameSize);
   LRU(pages, n, frameSize);
   Optimal(pages, n, frameSize);
   return 0;
}
```

OUTPUT:


"C:\Users\student\Desktop\1E
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
FIFO Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Total Page Faults = 9
LRU Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 3 1 2
Page 4 -> 3 4 2
Page 5 -> 3 4 5
Total Page Faults = 10
Optimal Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 1 2 4
Page 5 -> 1 2 5
Page 3 -> 3 2 5
Page 4 -> 4 2 5
Total Page Faults = 7
Process returned 0 (0x0) execution time : 36.677 s
Press any key to continue.